



The Bug Bounty Playbook: Authentication Bypass

The Bug Bounty Playbook · Part 3 of 20. 317 disclosed HackerOne reports analysed across 20 programs. Five recurring patterns. One testing...

Abhishek meena

Published August 1, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

The attack surface map	3
Pattern 1: Cookie scope theft via subdomain takeover	4
Pattern 2: OTP and 2FA logic flaws (the server verifies the wrong user)	5
Pattern 3: SSO, SAML, and OAuth protocol confusion	6
Pattern 4: Response manipulation and client-side verification	7
Pattern 5: Exposed infrastructure and token leakage	8
The escalation ladder	11
The report that made it click	11
The testing framework	12
What is next	13
Sources	13

The Bug Bounty Playbook · Part 3 of 20. 317 disclosed HackerOne reports analysed across 20 programs. Five recurring patterns. One testing framework you can run on every authentication endpoint.

30-second version

Authentication bypass is never about breaking the password. Across 317 top disclosed HackerOne reports, five patterns repeat: cookie scope theft via subdomain takeover, OTP and 2FA logic flaws, SSO and SAML protocol confusion, client-side response manipulation, and exposed infrastructure with missing auth. Every single one finds something the server trusts instead of the password and exploits that trust. The password was never the point.

The most upvoted auth bypass on HackerOne earned \$20,160 by reading cached plaintext passwords from a Pulse Secure VPN. The second most upvoted earned \$0 by changing one parameter in a Snapchat logout request. The third most upvoted earned \$0 by claiming an expired Roblox subdomain and stealing session cookies.

Three completely different techniques. Three completely different impact levels. One common thread: **none of them touched the password.**

I analysed 317 disclosed authentication bypass reports from HackerOne to understand why. I ranked them by community upvotes and bounty, read the top 35 in full detail across 20 programs, and extracted the patterns that repeat.

What I found is that authentication bypass is not about sophistication. It is about finding the one assumption the server makes that is wrong. Five assumptions appear across every program, every auth architecture, every bug class. Break any one of them and you log in as someone else.

The attack surface map

Every authentication bypass lives at the intersection of three things: an **identity claim** (who you say you are), a **proof** (what you bring as evidence), and a **verification step** that either checks the proof against the identity or skips the check entirely. The five patterns below are the five ways that intersection breaks down in production code.



Pattern 1: Cookie scope theft via subdomain takeover

This is the pattern that appeared at three completely different companies with the same chain.

Uber #219205 (181 upvotes, critical). The subdomain `saostatic.uber.com` pointed to Amazon CloudFront but the hostname was unregistered. The researcher claimed it. The SSO session cookie at `auth.uber.com` was scoped to `domain=.uber.com`. When a logged-in user visited the attacker-controlled subdomain, the browser sent the session cookie to the attacker's server. Full SSO bypass. Every `*.uber.com` service was compromised.

Roblox #335330 (780 upvotes, critical). Same chain. `devrel.roblox.com` pointed to an unclaimed Hubspot instance. The researcher claimed it. The `.ROBLOSECURITY` cookie was scoped to `*.roblox.com`. The attacker's page stole the cookie. But it went further: the subdomain was also whitelisted for CORS requests to `chat.roblox.com`, so the attacker could read private messages between users.

Ubiquiti #172137 (54 upvotes, high). Same chain again. `ping.ubnt.com` was an unclaimed CloudFront hostname. The `UBIC_AUTH` session cookie was scoped to `domain=.ubnt.com`. The researcher claimed the subdomain, generated a Let's Encrypt certificate, and stole session cookies from any user who visited the page.

Three companies. Three products. The same structural flaw: session cookies scoped to `*.domain.com` are only as safe as the least-secured subdomain.

How to test: Enumerate every subdomain. Check for unclaimed CNAME records pointing to CloudFront, Heroku, GitHub Pages, S3, or any CDN. If you can claim the subdomain, check whether the session cookie has `domain=.parent.com`. If it does, you have a full SSO bypass.

Pattern 2: OTP and 2FA logic flaws (the server verifies the wrong user)

Every 2FA bypass in the dataset is a logic flaw. None brute-force the code. None break the crypto. Every one finds a step where the server checks the right thing for the wrong user.

Snapchat #921780 (968 upvotes, critical). The endpoint `/scauth/otp/droid/logout` accepted a `user_id` in the JSON body. The server never checked that the `user_id` matched the authenticated session. The attacker logged in with their own credentials, sent a logout request with the victim's `user_id`, and the server responded with the victim's OTP token. They used that token to log in as the victim.

One parameter change. Full account takeover. 968 community upvotes. \$0 bounty.

Helium #810880 (\$100, 91 upvotes). The 2FA linking endpoint accepted a user ID in the request body. The attacker linked 2FA to the victim's account. Then they called `/api/2fa/verify` with the victim's user ID and a valid 2FA code. The server logged them in without a password. **Account takeover with only a user ID and a 2FA code.**

Localize #783258 (159 upvotes). The 2FA disable endpoint at `/api/user/two-factor/set` accepted any request with a valid session, regardless of whether the password was confirmed. An attacker with a stolen session cookie could disable 2FA without knowing the password.

HackerOne #2463279 (31 upvotes). Deactivate the account, reset the password via email, and 2FA is removed. The attacker needed email access, but the 2FA protection was supposed to prevent account takeover even with email access. The deactivation flow stripped 2FA without a separate verification step.

Nextcloud #1050244 (\$750, 63 upvotes). 2FA enforcement could be bypassed by logging in twice, stealing the `oc_sessionPassphrase` from the first session, and injecting it into the second. The enforcement check ran on session creation but not on session continuation.

The pattern is always the same: **the server verifies the code but not the user.** It checks that a valid 2FA code was submitted, but never checks that the user submitting it is the same user who started the login flow.

How to test: Map every step of the authentication flow as separate HTTP requests. For each step, ask: does the server verify that the user being verified is the same user who started the flow? Swap the user identifier between steps. If the server accepts a different user's identifier at any step, you have the bypass.

Pattern 3: SSO, SAML, and OAuth protocol confusion

SSO systems trust the protocol. The protocol is only as strong as its weakest implementation. Uber's SSO was bypassed three separate times for \$25,500 total, and each technique exploited a different protocol weakness.

Uber #136169 (\$10,000, 58 upvotes). The OneLogin SAML plugin on Uber's WordPress sites accepted a forged SAML response. The attacker crafted a SAML assertion with an arbitrary username, email, and role. The plugin created the user without verifying the signature against the IdP. The researcher gained administrator access on `newsroom.uber.com` by setting the role to "administrator".

Uber #138869 (\$7,000, 82 upvotes). The same OneLogin plugin created WordPress users with the default password `@@@nopass@@@`. The plugin blocked normal logins through `wp-login.php` but forgot to block XMLRPC. The attacker authenticated via XMLRPC using the default password and could create pages, upload files, and edit posts on every Uber WordPress site.

Uber #223014 (\$8,500, 87 upvotes). SAML authentication bypass on `uchat.uberinternal.com`. The SAML implementation accepted assertions that should have been rejected, allowing the attacker to bypass the SSO entirely and access the internal chat platform.

Twitter/Digits #126522 (126 upvotes). The Digits web authentication parsed query parameters differently on the client and server. The client split on `&` only. The server split on both `&` and `;`. The attacker appended `:@attacker.com` to the host parameter. The server saw a valid host. The client navigated to the attacker's domain. **OAuth credential theft through a parameter parsing mismatch.**

Nextcloud #3489490 (\$2,500, 48 upvotes). The OIDC implementation for ID4me handling did not verify the JWT signature. Any user could craft a JWT with an arbitrary identity claim and the server accepted it without checking the signature.

The community research from late 2025 confirms this pattern is expanding. The "Fragile Lock" research presented at Black Hat EU 2025 showed that SAML libraries are systemically broken at the parser level. Signature verification and assertion processing often use different XML parsers, and an attacker can inject a malicious assertion that the verifier ignores but the processor consumes. The

technique is called "Golden SAML" because you can reuse any IdP-signed document, even public metadata, to forge a response that passes signature validation.

The insight: SSO systems trust the protocol assertion. The protocol is only as strong as its weakest implementation. And the implementations are weaker than anyone assumed.

How to test: Fetch `/.well-known/openid-configuration` and `/.well-known/oauth-authorization-server` on every target. Test the SAML response: can you forge an assertion? Does the server verify the signature? What happens if you submit a SAML response signed by a different IdP? For OAuth, test `redirect_uri` with path traversal, parameter duplication, and `response_mode` changes. Check if dynamic client registration is enabled.

Pattern 4: Response manipulation and client-side verification

When auth validation happens client-side, the attacker is the server. The server sends a response, the client decides whether to proceed, and the attacker can modify the response in transit.

Twitter #770504 (287 upvotes, high). Twitter required a password reconfirmation before updating email or phone number. The server sent a response with a flow token and a success flag. The client checked the flag and either proceeded or showed an error. The attacker entered a wrong password, intercepted the response, and changed the success flag to `true` with the original flow token. The client proceeded. The attacker updated the victim's email and phone number without the password.

The server knew the password was wrong. The client never checked. **The gap between server-side knowledge and client-side trust is the bug.**

Hostinger #3081691 (212 upvotes, high). The `marketing.hostinger.com` subdomain was whitelisted for OAuth redirects but had an XSS vulnerability. The attacker crafted a redirect URL that executed JavaScript on the whitelisted domain. The script sent `window.location` (containing the auth token) to the attacker's server. The attacker used the stolen token to generate a JWT and take over the account. One click from the victim. Full account takeover.

Both reports exploit the same gap: the client trusts something the server sends, and the attacker can manipulate what the server sends. In Twitter's case, the attacker modifies the response. In Hostinger's case, the attacker injects content into the response. The authentication decision is made on the client side, and the client is the one component the attacker fully controls.

How to test: For every auth-related response, intercept it in your proxy. Change success flags, status codes, and tokens. If the client proceeds based on the modified response, the auth check is client-side only. For OAuth flows, check whether the redirect URL is on a whitelisted domain that has an XSS or open redirect. The redirect URL is where the token lives, and any vulnerability on that domain is a token theft.

Pattern 5: Exposed infrastructure and token leakage

Auth tokens and admin panels are never as hidden as you think. This pattern has the highest average bounty in the dataset because exposed infrastructure often leads directly to RCE.

LY Corporation #838635 (\$12,500, 233 upvotes). Spring Boot Actuator endpoints were publicly available with broken authentication. The `/actuator/env` endpoint leaked environment variables including database credentials. The `/actuator/heapdump` endpoint exposed the full heap, containing session tokens and internal secrets. From there, the researcher could access internal services and escalate to RCE.

Snapchat #258117 (117 upvotes). A test Jenkins instance had improper authentication. The researcher accessed the Jenkins script console, which allowed arbitrary Groovy execution. That is RCE. The instance was a test server, but it had access to the same internal network as production.

Mozilla #2915647 (123 upvotes, critical). A Netlify authentication token was exposed in public Mozilla CI logs on Taskcluster. The token had full access to the Mozilla IT Web SRE Netlify account, including owner, developer, and billing roles. The logs were publicly accessible by design. The token was sitting in a `live.log` file that anyone could read.

Slack #1378889 (\$3,500, 49 upvotes). The Slack Android app stored authentication tokens in a location accessible through a directory traversal vulnerability. An attacker could read auth tokens from the app's storage and impersonate the user. The tokens were stored with world-readable file permissions.

ownCloud #2337427 (\$2,000, 43 upvotes). The PreSignedURL feature had a flaw in its expiry validation. When the date was expired, the function returned a null error instead of denying access. The signing key was never checked. Anyone could access any file by constructing a URL with `OC-Signature=notchecked` and an expired date.

Rocket.Chat #3564655 (97 upvotes, critical). Complete authentication bypass to admin permissions. The authentication check could be skipped entirely, granting admin access without any credentials.

The pattern is simple: **if it exists on the internet without authentication, it is a finding**. Actuator endpoints, Jenkins instances, CI logs, auth tokens in mobile apps, PreSignedURL checks that return null. The attack surface is everything the auth system was supposed to protect but didn't.

How to test: Check for `/actuator`, `/admin`, `/jenkins`, `/phpmyadmin`, `/.git`, `/server-status`, and `/metrics` on every subdomain. Search CI logs for tokens, keys, and credentials. Check mobile apps for auth tokens stored with world-readable permissions. Test every PreSignedURL or token-based access mechanism for expired-but-accepted states.



The escalation ladder

The same auth bypass can be worth \$0 or \$12,500. The difference is the impact you demonstrate. Every report in the top 35 escalated beyond "I can bypass the login."

Rung 1: Token leakage. Credentials exposed in logs, CI, or mobile storage.

Mozilla #2915647 (123 upvotes, critical) found a Netlify token in public CI logs.

Slack #1378889 (\$3,500) found auth tokens through Android directory traversal.

The token itself is the finding. No bypass needed.

Rung 2: 2FA bypass. Log in as the victim without the second factor. Snapchat

#921780 (968 upvotes) did it with one parameter. Helium #810880 (\$100) did it by

linking 2FA to the victim's account. Localize #783258 (159 upvotes) did it by

disabling 2FA without password confirmation. The password is known or bypassed, and 2FA is the only remaining barrier.

Rung 3: Cookie scope theft. Steal the SSO session. Uber #219205 (181 upvotes,

critical), Roblox #335330 (780 upvotes, critical), and Ubiquiti #172137 (54

upvotes) all used subdomain takeover to steal session cookies. The victim never

knows. The attacker becomes the user across every service on the domain.

Rung 4: SSO/SAML/OAuth bypass. Impersonate any user, including admins.

Uber #136169 (\$10,000) forged a SAML response to create an admin. Uber

#138869 (\$7,000) used default passwords via XMLRPC. Uber #223014 (\$8,500)

bypassed SAML entirely. This is the rung where bounties cross \$10,000.

Rung 5: Infrastructure takeover. Admin access or RCE through exposed

infrastructure. LY Corp #838635 (\$12,500) reached RCE through Spring Actuator.

Snapchat #258117 (117 upvotes) reached RCE through Jenkins. Rocket.Chat

#3564655 (97 upvotes, critical) got admin with no credentials at all. This is the top of the ladder.

The report that made it click

The Snapchat OTP bypass (#921780, 968 upvotes) is the report I send to hunters who say "I know what auth bypass is, I just can't find them."

The researcher logged into their own Snapchat account. They went to the OTP logout endpoint at `/scauth/otp/droid/logout`. The endpoint accepted a `user_id` in the JSON body. The server used that `user_id` to determine whose OTP session to terminate.

The researcher replaced their own `user_id` with the victim's. The server responded with the victim's OTP token.

Then they called `/scauth/otp/login` with the victim's username and the OTP token they just received. The server logged them in as the victim.

What makes this report exceptional is the **insight**. The server trusted the `user_id` in the request body over the authenticated session. The session belonged to the attacker, but the server operated on the victim's account because the body said so. The server never asked: "Does this `user_id` match the session?"

This is the same fundamental flaw as the IDOR pattern from Part 1 of this series. In IDOR, the server trusts the object ID in the body over the session. In auth bypass, the server trusts the user ID in the body over the session. The bug is the same. The surface is different.

No scanner finds this. No automated check catches it. You find it by asking one question: **does the server verify that the `user_id` in the request matches the authenticated session?**

The testing framework

Run this on every authentication endpoint you encounter. Five questions, 60 seconds each:

1. **Does any auth endpoint accept a user identifier in the request body?** If the server trusts the body over the session, swap the `user_id`. If the server operates on the swapped user, you have a bypass. This is the Snapchat pattern.
2. ***Is the session cookie scoped to `.domain.com`?** If yes, every subdomain is an attack surface. Check for unclaimed subdomains. If you can claim one, the cookie is yours. This is the Uber pattern.
3. **Does the auth flow have multiple steps?** If yes, test skipping each step and swapping identifiers between steps. The server may verify step 1 but not step 2. This is the 2FA logic flaw pattern.
4. **Does the SSO/SAML/OAuth implementation verify signatures?** If not, forge an assertion. If it does, check which XML parser it uses. Parser discrepancies are where the bypasses live. This is the SAML pattern.
5. **Are there admin panels, actuator endpoints, or CI logs exposed without auth?** If yes, you have direct access. Check `/actuator`, `/jenkins`, `/.well-known/`, and CI log URLs for tokens and secrets. This is the infrastructure pattern.

Five questions. Every auth endpoint. That is the framework.

What is next

This is Part 3 of the Bug Bounty Playbook series. Each part takes one bug class, mines the top disclosed reports from HackerOne, and extracts the patterns you can actually test. Next in the series: **Part 4: Business Logic**, where the same methodology applied to the top business logic reports reveals a different kind of pattern: not broken auth or missing checks, but workflows that do the right thing in the wrong order.

If you want the framework as a checklist you can run on your next target, I have put it in a single page. Follow for the series.

Sources

1. Twitter VPN pre-auth RCE via Pulse Secure (\$20,160, 1239 upvotes) — hackerone.com/reports/591295 (https://hackerone.com/reports/591295)
2. Snapchat OTP logout bypass (968 upvotes, critical) — hackerone.com/reports/921780 (https://hackerone.com/reports/921780)
3. Roblox subdomain takeover to auth bypass (780 upvotes, critical) — hackerone.com/reports/335330 (https://hackerone.com/reports/335330)
4. Twitter password re-auth bypass via response manipulation (287 upvotes, high) — hackerone.com/reports/770504 (https://hackerone.com/reports/770504)
5. Starbucks auth cookie + path traversal (238 upvotes, critical) — hackerone.com/reports/876295 (https://hackerone.com/reports/876295)
6. LY Corporation Spring Actuator + broken auth (\$12,500, 233 upvotes, critical) — hackerone.com/reports/838635 (https://hackerone.com/reports/838635)
7. Hostinger 1-click ATO via auth token theft (212 upvotes, high) — hackerone.com/reports/3081691 (https://hackerone.com/reports/3081691)
8. GitHub auth bypass via SSH certificates (\$10,000, 181 upvotes) — hackerone.com/reports/1901040 (https://hackerone.com/reports/1901040)
9. Uber auth bypass via subdomain takeover of saostatic.uber.com (181 upvotes, critical) — hackerone.com/reports/219205 (https://hackerone.com/reports/219205)
10. TikTok ATO via auth bypass in account recovery (\$12,000, 169 upvotes, critical) — hackerone.com/reports/2443228 (https://hackerone.com/reports/2443228)
11. Localize 2FA disable without password (159 upvotes, medium) — hackerone.com/reports/783258 (https://hackerone.com/reports/783258)
12. Basecamp bot auth impersonation (\$2,000, 126 upvotes, high) — hackerone.com/reports/3329310 (https://hackerone.com/reports/3329310)
13. Twitter/Digits parameter parsing OAuth bypass (126 upvotes) — hackerone.com/reports/126522 (https://hackerone.com/reports/126522)

14. Mozilla Netlify auth token in public CI logs (123 upvotes, critical) — hackerone.com/reports/2915647 (https://hackerone.com/reports/2915647)
15. Snapchat RCE on test Jenkins via improper auth (117 upvotes) — hackerone.com/reports/258117 (https://hackerone.com/reports/258117)
16. Rocket.Chat complete auth bypass to admin (97 upvotes, critical) — hackerone.com/reports/3564655 (https://hackerone.com/reports/3564655)
17. Helium 2FA linking bypass (\$100, 91 upvotes) — hackerone.com/reports/810880 (https://hackerone.com/reports/810880)
18. Uber SAML auth bypass on uchat.uberinternal.com (\$8,500, 87 upvotes) — hackerone.com/reports/223014 (https://hackerone.com/reports/223014)
19. Uber OneLogin SAML bypass on WordPress (\$10,000, 58 upvotes) — hackerone.com/reports/136169 (https://hackerone.com/reports/136169)
20. Uber OneLogin XMLRPC auth bypass (\$7,000, 82 upvotes) — hackerone.com/reports/138869 (https://hackerone.com/reports/138869)
21. Ubiquiti auth bypass via subdomain takeover of ping.ubnt.com (54 upvotes, high) — hackerone.com/reports/172137 (https://hackerone.com/reports/172137)
22. Nextcloud 2FA enforcement bypass (\$750, 63 upvotes) — hackerone.com/reports/1050244 (https://hackerone.com/reports/1050244)
23. Slack Android directory traversal to auth token disclosure (\$3,500, 49 upvotes) — hackerone.com/reports/1378889 (https://hackerone.com/reports/1378889)
24. Nextcloud JWT signature verification bypass in OIDC (\$2,500, 48 upvotes) — hackerone.com/reports/3489490 (https://hackerone.com/reports/3489490)
25. ownCloud PreSignedURL auth bypass (\$2,000, 43 upvotes) — hackerone.com/reports/2337427 (https://hackerone.com/reports/2337427)
26. HackerOne 2FA bypass via account deactivation (31 upvotes) — hackerone.com/reports/2463279 (https://hackerone.com/reports/2463279)
27. Dataset: [reddelexc/hackerone-reports](https://github.com/reddelexc/hackerone-reports) (https://github.com/reddelexc/hackerone-reports) (317 auth bypass reports curated)